

required. Most contemporary performance evaluations involve the simulation of many benchmarks and a total of tens to hundreds of billion instructions. During the early phase of the design, performance simulators are used extensively to support the exploration of numerous tradeoffs, which require many simulation runs using different sets of parameters in the simulation model. For execution-driven performance simulators that have fairly detailed models of a complex machine, a slowdown factor of four to five orders of magnitude is rather common. In other words, to simulate a single machine cycle of the target machine, that is the machine being modeled, can require the execution of 10,000 to 100,000 machine cycles on the host machine. A large set of simulation runs can sometimes take many days to complete, even using a large pool of simulation machines.

1.4 Instruction-Level Parallel Processing

Instruction-level parallel processing can be informally defined as the concurrent processing of multiple instructions. Traditional sequential processors execute one instruction at a time. A leading instruction is completed before the next instruction is processed. To a certain extent, pipelined processors achieve a form of instruction-level parallel processing by overlapping the processing of multiple instructions. As many instructions as there are pipeline stages can be concurrently in flight at any one time. Traditional sequential (CISC) processors can require an average of about 10 machine cycles for processing each instruction, that is $CPI = 10$. With pipelined (RISC) processors, even though each instruction may still require multiple cycles to complete, by overlapping the processing of multiple instructions in the pipeline, the effective average CPI can be reduced to close to one if a new instruction can be initiated every machine cycle.

With scalar pipelined processors, there is still the limitation of fetching and initiating at most one instruction into the pipeline every machine cycle. With this limitation, the best possible CPI that can be achieved is one; or inversely, the best possible throughput of a scalar processor is one instruction per cycle (IPC). A more aggressive form of instruction-level parallel processing is possible that involves fetching and initiating multiple instructions into a wider pipelined processor every machine cycle. While the decade of the 1980s adopted $CPI = 1$ as its design objective for single-chip microprocessors, the goal for the decade of the 1990s was to reduce CPI to below one, or to achieve a throughput of IPC greater than one. Processors capable of IPC greater than one are termed *superscalar* processors. This section presents the overview of instruction-level parallel processing and provides the bridge between scalar pipelined processors and their natural descendants, the superscalar processors.

1.4.1 From Scalar to Superscalar

Scalar processors are pipelined processors that are designed to fetch and issue at most one instruction every machine cycle. *Superscalar* processors are those that are designed to fetch and issue multiple instructions every machine cycle. This subsection presents the basis and motivation for evolving from scalar to superscalar processor implementations.

1.4.1.1 Processor Performance. In Section 1.3.1 we introduced the iron law of processor performance, as shown in Equation (1.1). That equation actually represents the inverse of performance as a product of *instruction count*, average *CPI*, and the clock *cycle time*. We can rewrite that equation to directly represent performance as a product of the inverse of *instruction count*, average *IPC* ($IPC = 1/CPI$), and the clock *frequency*, as shown in Equation (1.2). Looking at this equation, we see that performance can be increased by increasing the IPC, increasing the frequency, or decreasing the instruction count.

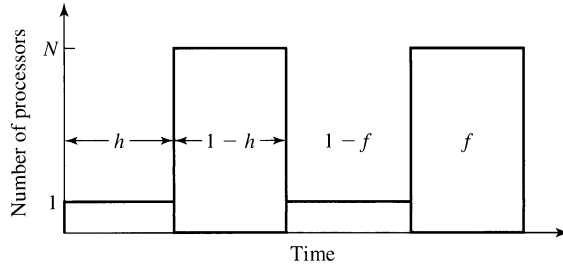
$$\text{Performance} = \frac{1}{\text{instruction count}} \times \frac{\text{instructions}}{\text{cycle}} \times \frac{1}{\text{cycle time}} = \frac{IPC \times \text{frequency}}{\text{instruction count}} \quad (1.2)$$

Instruction count is determined by three contributing factors: the instruction set architecture, the compiler, and the operating system. The ISA and the amount of work encoded into each instruction can strongly influence the total number of instructions executed for a program. The effectiveness of the compiler can also strongly influence the number of instructions executed. The operating system functions that are invoked by the application program effectively increase the total number of instructions executed in carrying out the execution of the program.

Average IPC (instructions per cycle) reflects the average instruction throughput achieved by the processor and is a key measure of microarchitecture effectiveness. Historically, the inverse of IPC, that is, CPI (cycles per instruction), has been used to indicate the average number of machine cycles needed to process each instruction. The use of CPI was popular during the days of scalar pipelined processors. The performance penalties due to various forms of pipeline stalls can be cleanly stated as different CPI overheads. Back then, the ultimate performance goal for scalar pipelined processors was to reduce the average CPI to one. As we move into the superscalar domain, it becomes more convenient to use IPC. The new performance goal for superscalar processors is to achieve an average IPC greater than one. The bulk of the microarchitecture techniques presented in this book target the improvement of IPC.

Frequency is strongly affected by the fabrication technology and circuit techniques. Increasing the number of pipeline stages can also facilitate higher clocking frequencies by reducing the number of logic gate levels in each pipe stage. Traditional pipelines can have up to 20 levels of logic gates in each pipe stage; most contemporary pipelines have only 10 or fewer levels. To achieve high IPC in superscalar designs, the pipeline must be made wider to allow simultaneous processing of multiple instructions in each pipe stage. The widening of the pipeline increases the hardware complexity and the signal propagation delay of each pipe stage. Hence, with a wider pipeline, in order to maintain the same frequency an even deeper pipeline may be required. There is a complex tradeoff between making pipelines wider and making them deeper.

1.4.1.2 Parallel Processor Performance. As we consider the parallel processing of instructions in increasing processor performance, it is insightful to revisit the classic observation on parallel processing commonly referred to as *Amdahl's law*

**Figure 1.5**

Scalar and Vector Processing in a Traditional Supercomputer.

[Amdahl, 1967]. Traditional supercomputers are parallel processors that perform both scalar and vector computations. During scalar computation only one processor is used. During vector computation all N processors are used to perform operations on array data. The computation performed by such a parallel machine can be depicted as shown in Figure 1.5, where N is the number of processors in the machine and h is the fraction of time the machine spends in scalar computation. Conversely, $1 - h$ is the fraction of the time the machine spends in vector computation.

One formulation of Amdahl's law states that the efficiency E of the parallel machine is measured by the overall utilization of the N processors or the fraction of time the N processors are busy. Efficiency E can be modeled as

$$E = \frac{h + N \times (1 - h)}{N} = \frac{h + N - Nh}{N} = 1 - h \times \left(1 - \frac{1}{N}\right) \quad (1.3)$$

As the number of processors N becomes very large, the efficiency E approaches $1 - h$, which is the fraction of time the machine spends in vector computation. As N becomes large, the amount of time spent in vector computation becomes smaller and smaller and approaches zero. Hence, as N becomes very large, the efficiency E approaches zero. This means that almost all the computation time is taken up with scalar computation, and further increase of N makes very little impact on reducing the overall execution time.

Another formulation of this same principle is based on the amount of work that can be done in the vector computation mode, or the *vectorizability* of the program. As shown in Figure 1.5, f represents the fraction of the program that can be parallelized to run in vector computation mode. Therefore, $1 - f$ represents the fraction of the program that must be executed sequentially. If T is the total time required to run the program, then the relative speedup S can be represented as

$$S = \frac{1}{T} = \frac{1}{(1-f) + (f/N)} \quad (1.4)$$

where T is the sum of $(1 - f)$, the time required to execute the sequential part, and f/N , the time required to execute the parallelizable part of the program. As N becomes very large, the second term of this sum approaches zero, and the total

execution time is dictated by the amount of time required to execute the sequential part. This is commonly referred to as the *sequential bottleneck*; that is, the time spent in sequential execution or scalar computation becomes a limit to how much overall performance improvement can be achieved via the exploitation of parallelism. As N increases or as the machine parallelism increases, the performance will become more and more sensitive to and dictated by the sequential part of the program.

The efficiency of a parallel processor drops off very quickly as the number of processors is increased. Furthermore, as the *vectorizability*, i.e., the fraction of the program that can be parallelized, of a program drops off slightly from 100%, the efficiency drop-off rate increases. Similarly the overall speedup drops off very quickly when f , the vectorizability of the program, drops even just very slightly from 100%. Hence, the overall performance improvement is very sensitive to the vectorizability of the program; or to state it another way, the overall speedup due to parallel processing is strongly dictated by the sequential part of the program as the machine parallelism increases.

1.4.1.3 Pipelined Processor Performance. Harold Stone proposed that a performance model similar to that for parallel processors can be developed for pipelined processors [Stone, 1987]. A typical execution profile of a pipelined processor is shown in Figure 1.6(a). The machine parallelism parameter N is now the depth of the pipeline, that is, the number of stages in the pipeline. There are three phases

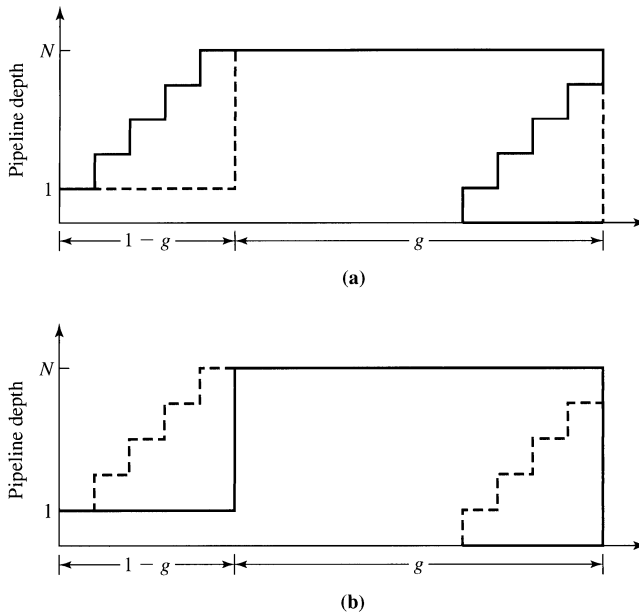


Figure 1.6

Idealized Pipelined Execution Profile: (a) Actual; (b) Modeled.

in this execution profile. The first phase is the pipeline filling phase during which the first sequence of N instructions enters the pipeline. The second phase is the pipeline full phase, during which the pipeline is full and represents the steady state of the pipeline. This is assuming that there is no pipeline disruption, and therefore represents the perfect pipeline execution profile. The third phase is the pipeline draining phase, during which no new instruction is entering the pipeline and the pipeline is finishing the instructions still present in the pipeline stages.

For modeling purposes, we can modify the execution profile of Figure 1.6(a) to the execution profile of Figure 1.6(b) by moving some of the work done during the pipeline filling phase to the pipeline draining phase. The total amount of work remains the same; that is, the areas within the two profiles are equal. The number of pipeline stages is N , the fraction of the time that all N pipeline stages are utilized is g , and $1 - g$ is the fraction of time when only one pipeline stage is utilized. Essentially $1 - g$ can be viewed as the fraction of time when only one instruction is moving through the pipeline; that is, there is no overlapping of instructions in the pipeline.

Unlike the idealized pipeline execution profile, the realistic pipeline execution profile must account for the stalling cycles. This can be done as shown in Figure 1.7(a). Instead of remaining in the pipeline full phase for the duration of the entire execution, this steady state is interrupted by pipeline stalls. Each stall effectively induces a new pipeline draining phase and a new pipeline filling phase, as shown in Figure 1.7(a), due to the break in the pipeline full phase. Similar modification can be performed on this execution profile to result in the modified profile of

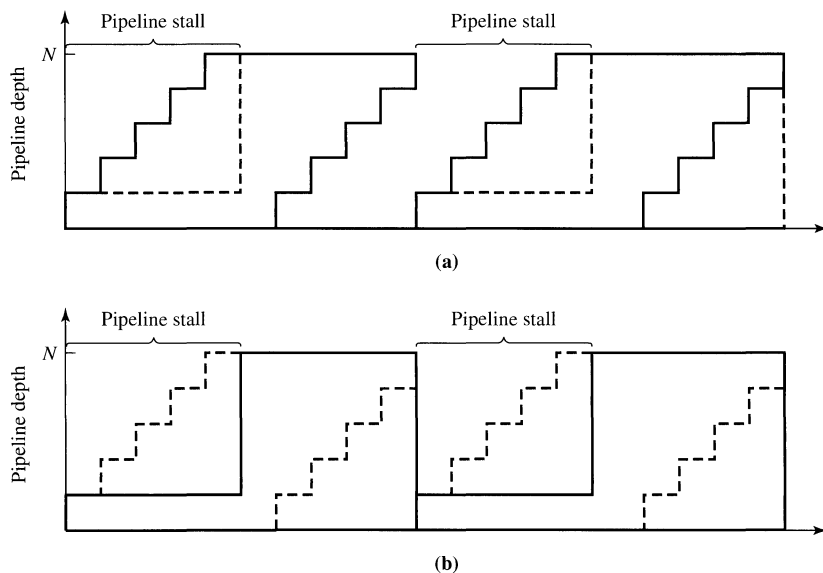


Figure 1.7

Realistic Pipeline Execution Profile: (a) Actual; (b) Modeled.

Figure 1.7(b) by moving part of the work done in the two pipeline filling phases to the two pipeline draining phases. Now the modified profile of Figure 1.7(b) resembles the execution profile of parallel processors as shown in Figure 1.5.

With the similarity of the execution profiles, we can now borrow the performance model of parallel processors and apply it to pipelined processors. Instead of being the number of processors, N is now the number of pipeline stages, or the maximum speedup possible. The parameter g now becomes the fraction of time when the pipeline is filled, and the parameter $1 - g$ now represents the fraction of time when the pipeline is stalled. The speedup S that can be obtained is now

$$S = \frac{1}{(1 - g) + (g/N)} \quad (1.5)$$

Note that g , the fraction of time when the pipeline is full, is analogous to f , the vectorizability of the program in the parallel processor model. Therefore, Amdahl's law can be analogously applied to pipelined processors. As g drops off just slightly from 100%, the speedup or the performance of a pipelined processor can drop off very quickly. In other words, the actual performance gain that can be obtained through pipelining can be strongly degraded by just a small fraction of stall cycles. As the degree of pipelining N increases, the fraction of stall cycles will become increasingly devastating to the actual speedup that can be achieved by a pipeline processor. Stall cycles in pipelined processors are now the key adversary and are analogous to the sequential bottleneck for parallel processors. Essentially, stall cycles constitute the pipelined processor's sequential bottleneck.

Equation (1.5) is a simple performance model for pipelined processors based on Amdahl's law for parallel processors. It is assumed in this model that whenever the pipeline is stalled, there is only one instruction in the pipeline, or it effectively becomes a sequential nonpipelined processor. The implication is that when a pipeline is stalled no overlapping of instructions is allowed; this is effectively equivalent to stalling the pipeline for N cycles to allow the instruction causing the stall to completely traverse the pipeline. We know, however, that with clever design of the pipeline, such as with the use of forwarding paths, to resolve a hazard that causes a pipeline stall, the number of penalty cycles incurred is not necessarily N and most likely less than N . Based on this observation, a refinement to the model of Equation (1.5) is possible.

$$S = \frac{1}{\frac{g_1}{1} + \frac{g_2}{2} + \dots + \frac{g_N}{N}} \quad (1.6)$$

Equation (1.6) is a generalization of Equation (1.5) and provides a refined model for pipelined processor performance. In this model, g_i represents the fraction of time when there are i instructions in the pipeline. In other words, g_i represents the fraction of time when the pipeline is stalled for $(N - i)$ penalty cycles. Of course, g_N is the fraction of time when the pipeline is full.

This pipelined processor performance model is illustrated by applying it to the six-stage TYP pipeline in Chapter 2. Note that the TYP pipeline has a load penalty

of one cycle and a branch penalty of four cycles. Based on the statistics from the IBM study presented in Chapter 2, the typical percentages of load and branch instructions are 25% and 20%, respectively. Assuming that the TYP pipeline is designed with a bias for a branch not taken, only 66.6% of the branch instructions, those that are actually taken, will incur the branch penalty. Therefore, only 13% of the instructions (branches) will incur the four-cycle penalty and 25% of the instructions (loads) will incur the one-cycle penalty. The remaining instructions (62%) will incur no penalty cycles. The performance of the TYP pipeline can be modeled as shown in Equation (1.7).

$$S_{TYP} = \frac{1}{\frac{0.13}{(6-4)} + \frac{0.25}{(6-1)} + \frac{0.62}{6}} = \frac{1}{\frac{0.13}{2} + \frac{0.25}{5} + \frac{0.62}{6}} = \frac{1}{0.22} = 4.5 \quad (1.7)$$

The resultant performance of the six-stage TYP pipeline processor is a factor of 4.5 over that of the sequential or nonpipelined processor. Note that the TYP is a six-stage pipeline with the theoretical speedup potential of 6. The actual speedup based on our model of Equation (1.6) is 4.5, as shown in Equation (1.7), which can be viewed as the *effective degree of pipelining* of the TYP pipeline. Essentially the six-stage TYP processor behaves as a perfect pipeline with 4.5 pipeline stages. The difference between 6 and 4.5 reflects the difference between the potential (peak) pipeline parallelism and the achieved (actual) pipeline parallelism.

1.4.1.4 The Superscalar Proposal. We now restate Amdahl's law that models the performance of a parallel processor:

$$S = \frac{1}{(1-f) + (f/N)} \quad (1.8)$$

This model gives the performance or speedup of a parallel system over that of a nonparallel system. The *machine parallelism* is measured by N , the number of processors in the machine, and reflects the maximum number of tasks that can be simultaneously performed by the system. The parameter f , however, is the vectorizability of the program which reflects the *program parallelism*. The formulation of this model is influenced by traditional supercomputers that contain a scalar unit and a vector unit. The vector unit, consisting of N processors, executes the vectorizable portion of the program by performing N tasks at a time. The nonvectorizable portion of the program is then executed in the scalar unit in a sequential fashion. We have already observed the oppressive tyranny of the nonvectorizable portion of the program on the overall performance that can be obtained through parallel processing.

The assumption that the nonvectorizable portion of the program must be executed sequentially is overly pessimistic and not necessary. If some, even low, level of parallelism can be achieved for the nonvectorizable portion of the program, the severe impact of the sequential bottleneck can be significantly moderated. Figure 1.8 illustrates this principle. This figure, taken from an IBM technical report coauthored

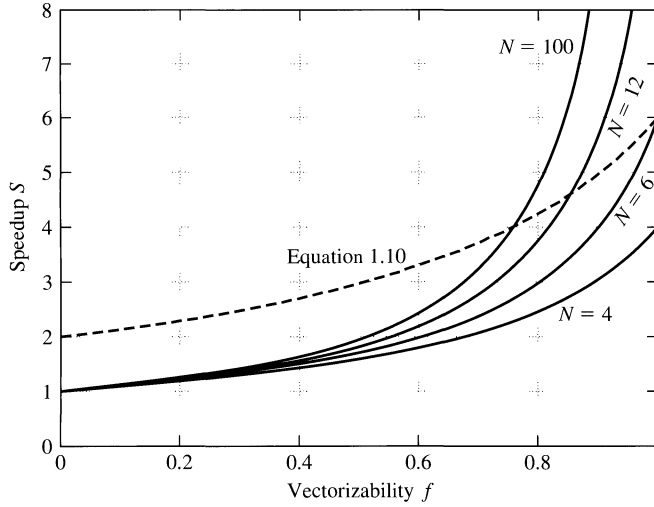


Figure 1.8

Easing of the Sequential Bottleneck with Instruction-Level Parallelism for Nonvectorizable Code.

Source: Agerwala and Cocke, 1987.

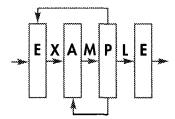
by Agerwala and Cocke [1987], plots the speedup as a function of f , the vectorizability of a program, for several values of N , the maximum parallelism of the machine. Take the example of the case when $N = 6$. The speedup is

$$S = \frac{1}{(1-f) + (f/6)} \quad (1.9)$$

Examining the curve for Equation (1.9) in Figure 1.8, we see that the speedup is equal to 6 if f is 100%, that is, perfectly vectorizable. As f drops off from 100%, the speedup drops off very quickly; as f becomes 0%, the speedup is one; that is, no speedup is obtained. With higher values of N , this speedup drop-off rate gets significantly worse, and as f approaches 0%, all the speedups approach one, regardless of the value of N . Now assume that the minimum degree of parallelism of 2 can be achieved for the nonvectorizable portion of the program. The speedup now becomes

$$S = \frac{1}{\frac{(1-f)}{2} + \frac{f}{6}} \quad (1.10)$$

Examining the curve for Equation (1.10) in Figure 1.8, we see that it also starts at a speedup of 6 when f is 100%, but drops off more slowly than the curve for Equation (1.9) when f is lowered from 100%. In fact this curve crosses over the curve for Equation (1.8) with $N = 100$ when f is approximately 75%. This means that for cases with f less than 75%, it is more beneficial to have a system with maximum parallelism of only 6, that is $N = 6$, but a minimum parallelism of two for the nonvectorizable portion, than a system with maximum parallelism of $N = 100$ with

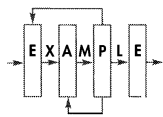


sequential execution of the nonvectorizable portion. The vectorizability of a program f is a complex function involving the application algorithm, the programming language, the compiler, and the architecture. Other than those for scientific applications involving mostly numerical computations, most programs for general-purpose computing tend not to have very high vectorizability. It is safe to say that most general-purpose programs have f less than 75%, and for many, significantly less.

One primary motivation for designing superscalar processors is to develop general-purpose processors that can achieve some (perhaps low relative to vectorizing) level of parallelism for a wide range of application programs. The goal is to ensure that some degree of instruction-level parallelism can be achieved for all portions of the program so as to moderate the severe impact of the sequential bottleneck. Of course, the highly vectorizable programs will continue to achieve good speedup via parallelism. Note that the curve for Equation (1.10) is always higher than that for Equation (1.9) even at high values of f , and is higher than other curves for large values of N at lower values of f . The goal for superscalar processors is to achieve generalized instruction-level parallelism and the consequent speedup for all types of application programs, including those that are not necessarily vectorizable.

1.4.2 Limits of Instruction-Level Parallelism

In Equation (1.10), parallelism of degree 6 can be achieved for the f fraction of the program and parallelism of degree 2 can be achieved for the remaining $1 - f$ fraction of the program. The speedup S can be viewed as the *aggregate degree of parallelism* that can be achieved for the entire program. For example, if the parameter f is 50% and the peak parallelism N is 6, then the speedup or the aggregate degree of parallelism is



$$S = \frac{1}{\frac{(1-f)}{2} + \frac{f}{6}} = \frac{1}{\frac{0.5}{2} + \frac{0.5}{6}} = 3 \quad (1.11)$$

The implication of Equation (1.11) is that effectively an overall or aggregate degree of parallelism of 3 is achieved for the entire program. Applying this result at the instruction level, we see that Equation (1.11) indicates that an average of three instructions can be simultaneously executed at a time. For traditional vector computation, the number of operations that can be simultaneously performed is largely determined by the size of the vectors or arrays, or essentially the data set size. For general-purpose unstructured programs, the key question is, what aggregate degree of instruction-level parallelism can potentially be achieved?

Instruction-level parallelism can be informally defined as the aggregate degree of parallelism (measured by the number of instructions) that can be achieved by the concurrent execution of multiple instructions. Possible limits of ILP have been investigated for almost three decades. Numerous experimental studies have been performed that yield widely varying results on purported limits of ILP. The following table provides a sample listing of reported limits in order of increasing degrees of ILP.

Study	ILP Limit
Weiss and Smith, 1984	1.58
Sohi and Vajapeyam, 1987	1.81
Tjaden and Flynn, 1970	1.86
Tjaden and Flynn, 1973	1.96
Uht and Wedig, 1986	2.0
Smith et al., 1989	2.0
Jouppi and Wall, 1989	2.4
Johnson, 1991	2.5
Acosta et al., 1986	2.79
Wedig, 1982	3.0
Butler et al., 1991	5.8
Melvin and Patt, 1991	6
Wall, 1991	7
Kuck et al., 1972	8
Riseman and Foster, 1972	51
Nicolau and Fisher, 1984	90

This listing is certainly not exhaustive, but clearly illustrates the diversity and possible inconsistency of the research findings. Most of these are limit studies making various idealized assumptions. The real challenge is how to achieve these levels of ILP in realistic designs. The purported limits are also not monotonic with respect to chronological order. During the decade of the 1990s the debate on the limits of ILP replaced the RISC vs. CISC debate of the 1980s [Colwell et al., 1985]. This new debate on the limit of ILP is still not settled.

1.4.2.1 Flynn's Bottleneck. One of the earliest studies done at Stanford University by Tjaden and Flynn in 1970 concluded that the ILP for most programs is less than 2. This limit has been informally referred to as *Flynn's bottleneck*. This study focused on instruction-level parallelism that can be found within basic block boundaries. Since crossing basic block boundaries involves crossing control dependences, which can be dependent on run-time data, it is assumed that the basic blocks must be executed sequentially. Because of the small size of most basic blocks, typically the degree of parallelism found is less than 2. This result or Flynn's bottleneck has since been confirmed by several other studies.

One study in 1972 that confirmed this result was by Riseman and Foster [1972]. However, they extended their study to examine the degree of ILP that can be achieved if somehow control dependences can be surmounted. This study reported various degrees of parallelism that can be achieved if various numbers of control dependences can be overcome. If the number of control dependences that can be overcome is unlimited, then the limit of ILP is around 51. This study highlights the strong influence of control dependences on the limits of ILP.

1.4.2.2 Fisher's Optimism. At the other end of the spectrum is a study performed by Nicolau and Fisher in 1984 at Yale University. This study hints at almost unlimited amounts of ILP in many programs. The benchmarks used in this study tend to be more numerical, and some of the parallelisms measured were due to data parallelism resulting from array-type data sets. An idealized machine model capable of executing many instructions simultaneously was assumed. While some idealized assumptions were made in this study, it does present a refreshing optimistic outlook on the amount of ILP that can be harvested against the pessimism due to Flynn's bottleneck. We informally refer to this purported limit on ILP as *Fisher's optimism*.

Initially this optimism was received with a great deal of skepticism. A number of subsequent events somewhat vindicated this study. First a prototype machine model called the VLIW (very long instruction word) processor was developed along with a supporting compiler [Fisher, 1983]. Subsequently, a commercial venture (Multiflow, Inc.) was formed to develop a realistic VLIW machine, which resulted in the Multiflow TRACE computer. The TRACE machines were supported by a powerful VLIW compiler that employs *trace scheduling* (developed by Josh Fisher et al.) to extract instruction-level parallelism [Fisher, 1981]. Multiflow, Inc., was reasonably successful and eventually had an installed base of more than 100 machines. More importantly, the short-lived commercial TRACE machines were the first general-purpose uniprocessors to achieve an average IPC greater than one. Although the actual levels of ILP achieved by the TRACE machines were far less than the limits published earlier by Nicolau and Fisher in 1984, they did substantiate the claims that there are significant amounts of ILP that can be harvested beyond the previously accepted limit of 2.

1.4.2.3 Contributing Factors. Many of the studies on the limits of ILP employ different experimental approaches and make different assumptions. Three key factors contribute to the wide range of experimental results: benchmarks used, machine models assumed, and compilation techniques employed. Each study adopts its own set of benchmarks, and frequently the results are strongly influenced by the benchmarks chosen. Recently, the Standard Performance Evaluation Corporation (SPEC) benchmark suites have become widely adopted, and most manufacturers of processors and computing systems provide SPEC ratings for their systems. While strict guidelines exist for manufacturers to report the SPEC ratings on their products (see www.spec.org), there are still quite nonuniform uses of the SPEC ratings by researchers. There are also strong indications that the SPEC benchmark suites are only appropriate for workstations running scientific and engineering applications, and are not relevant for other application domains such as commercial transaction processing and embedded real-time computing.

The second key factor that contributes to the confusion and controversy on the limits of ILP is the assumptions made by the various studies about the machine model. Most of the limit studies assume idealized machine models. For example, the cache memory is usually not considered or is assumed to have a 100% hit rate with one-cycle latency. Some models assume infinite-sized machines with infinite register files. Usually one-cycle latency is assumed for all operation and functional

unit types. Other studies employ more realistic machine models, and these usually resulted in more pessimistic, and possibly unnecessarily pessimistic, limits. Of course, there is also a great deal of nonuniformity in the instruction set architectures used. Some are fictitious architectures, and others use existing architectures. The architectures used also tend to have a strong influence on the experimental results.

Finally, the assumptions made about the compilation techniques used are quite diverse. Many of the studies do not include any consideration about the compiler; others assume infinitely powerful compilers. Frequently, these studies are based on dynamic traces collected on real machines. Simulation results based on such traces are not only dependent on the benchmarks and architectures chosen, but also strongly dependent on the compilers used to generate the object code. The potential contribution of the compilation techniques to the limits of ILP is an ongoing area of research. There is currently a significant gap between the assumed capabilities of all-powerful compilers and the capabilities of existing commercially available compilers. Many anticipate that many more advancements can be expected in the compilation domain.

Probably the safest conclusion drawn from the studies done so far is that the real limit of ILP is beyond that being achieved on current machines. There is room for more and better research. The assumption of any specific limit is likely to be premature. As more powerful and efficient microarchitectures are designed and more aggressive compilation techniques are developed, the previously made assumptions may have to be changed and previously purported limits may have to be adjusted upward.

1.4.3 Machines for Instruction-Level Parallelism

Instruction-level parallelism is referred to as *fine-grained parallelism* relative to other forms of coarse-grained parallelism involving the concurrent processing of multiple program fragments or computing tasks. Machines designed for exploiting general ILP are referred to as ILP machines and are typically uniprocessors with machine resource parallelisms at the functional unit level. A classification of ILP machines was presented by Norm Jouppi in 1989 [Jouppi and Wall, 1989]. ILP machines are classified according to a number of parameters.

- *Operation latency (OL)*. The number of machine cycles until the result of an instruction is available for use by a subsequent instruction. The reference instruction used is a simple instruction that typifies most of the instructions in the instruction set. The operation latency is the number of machine cycles required for the execution of such an instruction.
- *Machine parallelism (MP)*. The maximum number of simultaneously executing instructions the machine can support. Informally, this is the maximum number of instructions that can be simultaneously in flight in the pipeline at any one time.
- *Issue latency (IL)*. The number of machine cycles required between issuing two consecutive instructions. Again the reference instructions are simple

instructions. In the present context, *issuing* means the initiating of a new instruction into the pipeline.

- *Issue parallelism (IP)*. The maximum number of instructions that can be issued in every machine cycle.

In Jouppi's classification, the scalar pipelined processor is used as the baseline machine. The classification also uses a generic four-stage instruction pipeline for illustration. These stages are

1. IF (instruction fetch)
2. DE (instruction decode)
3. EX (execute)
4. WB (write back)

The EX stage is used as a reference for the determination of the operation latency. The scalar pipelined processor, used as the baseline machine, is defined to be a machine with $OL = 1$ cycle and $IL = 1$ cycle. This baseline machine, with its instruction processing profile illustrated in Figure 1.9, can issue one new instruction into the pipeline in every cycle, and a typical instruction requires one machine cycle for its execution. The corresponding MP is equal to k , the number of stages in the pipeline; in Figure 1.9 $MP = 4$. The IP is equal to one instruction per cycle. Notice all four of these parameters are static parameters of the machine and do not take into account the dynamic behavior that depends on the program being executed.

When we discuss the performance or speedup of ILP machines, this baseline machine is used as the reference. Earlier in this chapter we referred to the speedup that can be obtained by a pipelined processor over that of a sequential nonpipelined processor that does not overlap the processing of multiple instructions. This form of speedup is restricted to comparison within the domain of scalar processors and focuses on the increased throughput that can be obtained by a (scalar) pipelined processor with respect to a (scalar) nonpipelined processor. Beginning with Chapter 3,

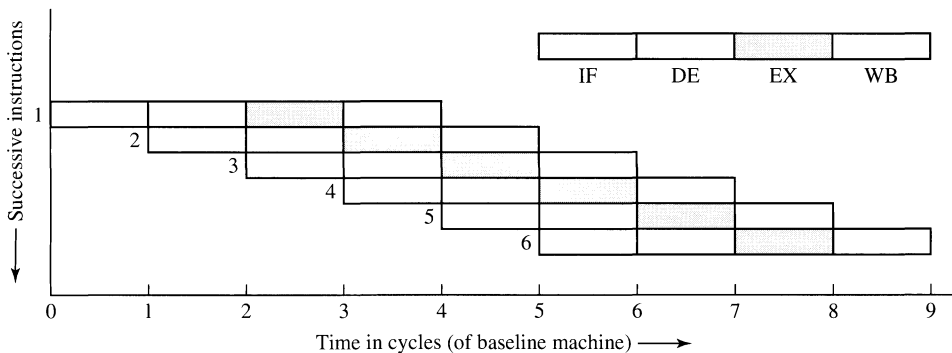


Figure 1.9

Instruction Processing Profile of the Baseline Scalar Pipelined Machine.

which deals with ILP machines, the form of speedup referred to is the performance of an ILP machine compared to the scalar pipelined processor, which is used as the new reference machine.

1.4.3.1 Superpipelined Machines. A *superpipelined* machine is defined with respect to the baseline machine and is a machine with higher degrees of pipelining than the baseline machine. In a superpipelined machine, the machine cycle time is shorter than that of the baseline machine and is referred to as the *minor cycle* time. The cycle time of a superpipelined machine is $1/m$ of the baseline cycle time, or equivalently there are m minor cycles in the baseline cycle. A superpipelined machine is characterized by $OL = 1 \text{ cycle} = m \text{ minor cycles}$ and $IL = 1 \text{ minor cycle}$. In other words, the simple instruction still requires one baseline cycle, equal to m minor cycles, for execution, but the machine can issue a new instruction in every minor cycle. Consequently, $IP = 1 \text{ instruction/minor cycle} = m \text{ instructions/cycle}$, and $MP = m \times k$. The instruction processing profile of a superpipelined machine is shown in Figure 1.10.

A superpipelined machine is a pipelined machine in which the degree of pipelining is beyond that dictated by the operation latency of the simple instructions. Essentially superpipelining involves pipelining of the execution stage into multiple stages. An “underpipelined” machine cannot issue instructions as fast as they are executed. On the other hand, a superpipelined machine issues instructions faster than they are executed. A superpipelined machine of degree m , that is, one that takes m minor cycles to execute a simple operation, can potentially achieve better performance than that of the baseline machine by a factor of m . Technically, traditional pipelined computers that require multiple cycles for executing simple operations should be classified as superpipelined. For example, the latency for performing fixed-point addition is three cycles in both the CDC 6600 [Thornton, 1964] and the CRAY-1 [Russell, 1978], and new instructions can be issued in every cycle. Hence, these are really superpipelined machines.

In a way, the classification of superpipelined machines is somewhat artificial, because it depends on the choice of the baseline cycle and the definition of a simple operation. The key characteristic of a superpipelined machine is that the

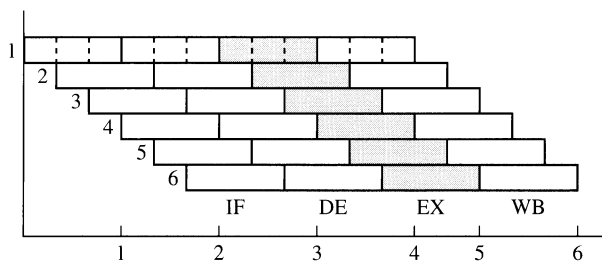


Figure 1.10

Instruction Processing Profile of a Superpipelined Machine of Degree $m = 3$.

result of an instruction is not available to the next $m - 1$ instructions. Hence, a superpipelined processor can be viewed simply as a more deeply pipelined processor with some restrictions on the placement of forwarding paths. In a standard pipelined processor, the implicit assumption is that the sources and destinations of forwarding paths can be the outputs and inputs, respectively, of any of the pipeline stages. If a superpipelined machine is viewed as a deeply pipelined machine with $m \times k$ stages, then the outputs of some of the stages cannot be accessed for forwarding and the inputs of some of the stages cannot receive forwarded data. The reason for this is that some of the operations that require multiple minor cycles and multiple pipeline stages to complete are primitive operations, in the sense of being noninterruptible for the purpose of data forwarding. This is really the key distinction between pipelined and superpipelined machines. In this book, outside of this section, there is no special treatment of superpipelined machines as a separate class of processors distinct from pipelined machines.

The 64-bit MIPS R4000 processor is one of the first processors claimed to be “superpipelined.” Internally, the R4000 has eight physical stages in its pipeline, as shown in Figure 1.11, with a physical machine cycle time of 10 nanoseconds (ns) [Bashteen et al., 1991, Mirapuri et al., 1992]. However, the chip requires a 50-MHz clock input and has an on-chip clock doubler. Consequently, the R4000 uses 20 ns as its baseline cycle, and it is considered superpipelined of degree 2 with respect to a four-stage baseline machine with a 20-ns cycle time. There are two minor cycles to every baseline cycle. In the case of the R4000, the multicycle primitive operations are the cache access operations. For example, the first two physical stages (IF and IS) are required to perform the I-cache access, and similarly the DF and DS physical stages are required for D-cache access. These are noninterruptible operations; no data forwarding can involve the buffers between the IF and IS stages or the buffers between the DF and DS stages. Cache accesses, here considered “simple” operations, are pipelined and require an operation latency of two (minor) cycles. The issue latency for the entire pipeline is one (minor) cycle; that is, one new instruction can be issued every

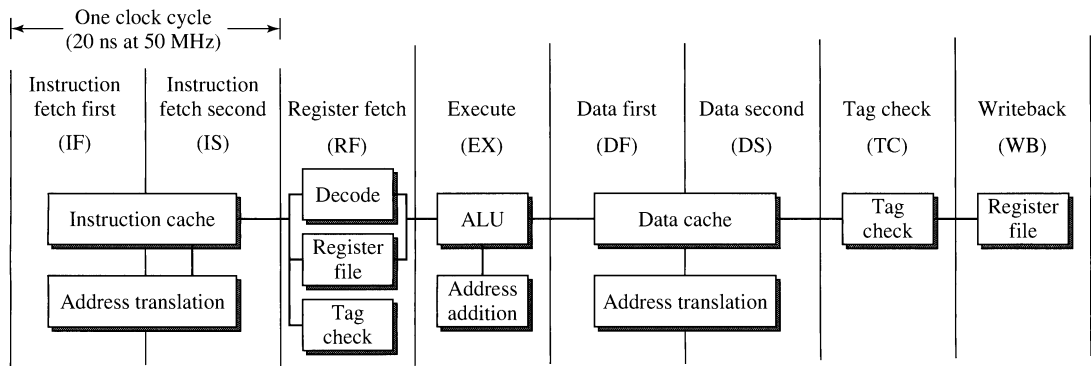


Figure 1.11
The “Superpipelined” MIPS R4000 8-Stage Pipeline.

10 ns. Potentially the R4000 can achieve a speedup over the baseline four-stage pipeline by a factor of 2.

1.4.3.2 Superscalar Machines. Superscalar machines are extensions of the baseline scalar pipelined machines and are characterized by $OL = 1$ cycle, $IL = 1$ cycle, and $IP = n$ instructions/cycle. The machine cycle is the same as the baseline cycle; there are no minor cycles. A simple operation is executed in one cycle. In every cycle, multiple instructions can be issued. The superscalar degree is determined by the issue parallelism n , the maximum number of instructions that can be issued in every cycle. The instruction processing profile of a superscalar machine is illustrated in Figure 1.12. Compared to a scalar pipelined processor, a superscalar machine of degree n can be viewed as having n pipelines or a pipeline that is n times wider in the sense of being able to carry n instructions in each pipeline stage instead of one. A superscalar machine has $MP = n \times k$. It has been shown that a superpipelined machine and a superscalar machine of the same degree have the same machine parallelism and can achieve roughly the same level of performance.

There is no reason why a superscalar machine cannot also be superpipelined. The issue latency can be reduced to $1/m$ of the (baseline) cycle while maintaining the issue parallelism of n instructions in every (minor) cycle. The total issue parallelism or throughput will be $n \times m$ instructions per (baseline) cycle. The resultant machine parallelism will become $MP = n \times m \times k$, where n is the superscalar degree, m is the superpipelined degree, and k is the degree of pipelining of the baseline machine. Alternatively the machine parallelism can be viewed as $MP = n \times (m \times k)$, representing a superscalar machine with $m \times k$ pipeline stages. Such a machine can be equivalently viewed as a more deeply pipelined processor of $m \times k$ stages with superscalar degree n , without having to invoke the tedious term “superscalar-superpipelined” machine; and we won’t.

1.4.3.3 Very-Long-Instruction-Word Machines. Quite similar to the superscalar machines is another class of ILP machines called VLIW (very long instruction word) machines by Josh Fisher [Fisher, 1983]. The intent and performance objectives are very similar for these two classes of machines; the key difference lies in the

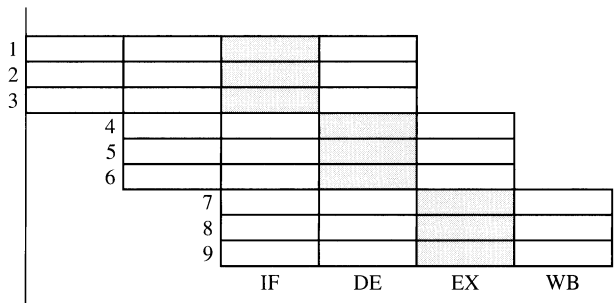
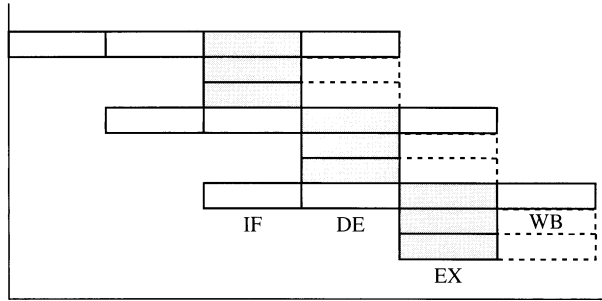


Figure 1.12

Instruction Processing Profile of a Superscalar Machine of Degree $n = 3$.

**Figure 1.13**

Instruction Processing Profile of a VLIW Machine of Degree $n = 3$.

placement of the dynamic-static interface (DSI) or the partitioning of what is done at run time via hardware mechanisms and what is done at compile time via software means. The instruction processing profile of a VLIW machine is illustrated in Figure 1.13.

Unlike in a superscalar machine, the IF and DE stages of a VLIW machine need not be replicated to support the simultaneous processing, that is, fetching and decoding, of n separate instructions. In a superscalar machine, the decision of which n instructions are to be issued into the execute stage is made at run time. For a VLIW machine, such an instruction-issuing decision is made at compile time, and the n instructions to be simultaneously issued into the execute stage are determined by the compiler and stored appropriately in the program memory as a very long instruction word.

Superscalar and VLIW machines represent two different approaches to the same ultimate goal, which is achieving high processor performance via instruction-level parallel processing. The two approaches have evolved through different historical paths and from different perspectives. It has been suggested that these two approaches are quite synergistic, and there is a strong motivation for pursuing potential integration of the two approaches. This book focuses on dynamic techniques implemented in the microarchitecture; hence, we will not address in depth VLIW features that rely on aggressive compile-time techniques.

1.5 Summary

Microprocessors have had an unparalleled impact on the computer industry. The changes that have taken place during the lifetime of microprocessors (30+ years) have been phenomenal. Microprocessors are now entering their fourth decade. It is fascinating to speculate on what we can expect from microprocessors in this coming decade.

Although it was the fad of past decades, instruction set architecture (ISA) design is no longer a very interesting topic. We have learned a great deal about how to design an elegant and scalable ISA. However, code compatibility and the software installed base are more crucial in determining the longevity of an ISA. It has been amply shown that any ISA deficiency can be overcome by microarchitecture

techniques. Furthermore, with the emergence of portable bytecodes and dynamic just-in-time (JIT) compilation, the meaning of ISA and the consequent placement of the dynamic-static interface (DSI) will become quite blurred.

In the coming decade, microarchitecture will be where the action is. As the chip integration density approaches 1 billion transistors on a die, many of the traditional (macro)architecture features, such as the memory subsystem, multiple processors, and input/output subsystem, will become on-die issues and hence become effectively microarchitecture issues. Traditional system-level architecture will become part of chip-level design. We can expect to see the integration of multiple processors, the cache memory hierarchy, the main memory controller (and possibly even the main memory), input/output devices, and network interface devices on one chip.

We can expect to see many new innovative microarchitecture techniques. As we approach and possibly exceed the 10-GHz clocking speed, we will need to rethink many of the fundamentals of microarchitecture design. A simple ALU operation may take multiple cycles. A sophisticated branch predictor can require up to 10 cycles. Main memory latency will be 1000+ cycles long. It may take tens of clock cycles to traverse the entire die. What we currently think of as very aggressive pipelining will be viewed as rather elementary.

Future microprocessors will become single-chip computing systems that will need to exploit various forms of parallelism. These systems will need to go beyond instruction-level parallelism to harvest thread-level parallelism (TLP) in the workload. Perhaps the most important will be the pursuit of memory-level parallelism (MLP) in being able to process many simultaneous memory accesses. As main memory latency becomes three orders of magnitude slower than the CPU cycle time, we will need to find clever ways of trading the memory bandwidth to mitigate the severe negative impact of long memory latency on overall performance. The main challenge will become the movement of data, not the operations performed on the data.

REFERENCES

- Acosta, R., J. Kilestrup, and H. Torng: "An instruction issuing approach to enhancing performance in multiple functional unit processors," *IEEE Trans. on Computers*, C35, 9, 1986, pp. 815–828.
- Agerwala, T., and J. Cocke: "High performance reduced instruction set processors," Technical report, IBM Computer Science, 1987.
- Amdahl, G.: "Validity of the single processor approach to achieving large scale computing capabilities," *AFIPS Conf. Proc.*, 1967, pp. 483–485.
- Amdahl, G., G. Blaauw, and F. P. Brooks, Jr.: "Architecture of the IBM System/360," *IBM Journal of Research and Development*, 8, 1964, pp. 87–101.
- Bashteen, A., I. Lui, and J. Mullan: "A superpipeline approach to the MIPS architecture," *Proc. COMPCON Spring 91*, 1991, pp. 325–333.
- Blaauw, G., and F. P. Brooks, Jr.: *Computer Architecture: Concepts and Evolution*. Reading, MA: Addison-Wesley, 1997.
- Black, B., and J. P. Shen: "Calibration of microprocessor performance models," *Computer*, 31, 5, 1998, pp. 59–65.